

Shor's Factoring Algorithm

 Copy page



View on GitHub

Open this notebook in GitHub to run it yourself

Integer Factorization [1] is a famous problem in number theory: given a composite number N , find its prime factors. The importance of the problem stems from there being no known efficient classical algorithm (in the number of bits needed to represent N in polynomial time), and much of modern-day cryptography relies on this fact. In 1994, Peter Shor developed an efficient quantum algorithm for the problem [2], providing arguably the most important evidence for an exponential advantage of quantum computing over classical computing.

- **Input:** A composite integer N .
- **Promise:** N is not a prime power and has at least one nontrivial factor.
- **Output:** With high probability, a nontrivial factor of N .

Complexity: Runs in $\text{polylog}(N)$ time, i.e., polynomial in $\log N$, giving an exponential speedup over the best-known classical factoring algorithms.

Keywords: Integer factoring, Order-finding, Period finding, Phase estimation, Hidden subgroup over $\mathbb{Z}$, RSA/ECC break, Cryptography, Exponential speedup, Modular exponentiation.

Shor's algorithm consists of classical parts and a quantum subroutine.

The notebook is organized as follows: First, as an introduction, we outline the steps of the algorithm for factoring an input number N , summarized from [3]. Then, we construct all the classical and

The steps of Shor's algorithm:

1. Pick a random number $1 < a < N$ that is co-prime with N .

Check co-primality by computing the GCD (greatest common divisor) of a and N . If it is 1, then we have a co-prime a ; otherwise, we have a non-trivial factor of N , and we are done (the GCD complexity is $O(\log(N))$).

1. Find the period r of the following function, using the quantum period finding algorithm:

$$f(x) = a^x \bmod N$$

3. If r is odd or $a^{r/2} = -1 \bmod N$, return to step 1 (this event can be shown to happen with probability at most $1/2$).
4. Otherwise, $\gcd(a^{r/2} \pm 1, N)$ are both factors of N , and computing one of them yields the required result.

Next, we move to the implementation of each and every step. *The quantum part is designed naturally as a Quantum Phase Estimation (QPE) routine, using Classiq's `flexible_qpe` and `Modular Arithmetic`.*

Finding a Co-Prime of N

This is a simple classical preprocess for the algorithm. We randomly pick an integer a in $(1, N)$, and check whether it is a co-prime of N by calling the Euclidian GCD algorithm. If the sampled integer is not a co-prime then we find a factor of N and we are done. (When we run the algorithm for a specific example below, we will fix a co-prime a in advance, such that we have a meaningful demo with a quantum part).

```
"""
Draw an integer in (1,N), and check if it is a co-prime of N. If True, return
"""
while True:
    a = np.random.randint(2, N)
    gcd_a = np.gcd(a, N)
    if gcd_a == 1:
        return int(a), gcd_a
    else:
        print(f"The number {N} is factored by {np.gcd(a, N)} and {N//np.gcd(a, N)}")
        return int(a), gcd_a
```

We can run a simple example:

```
N = 123456789
a, gcd_a = random_coprime(N)
print(f"The numbers {a} and {N} have the greatest common divisor {gcd_a}")
```

 Output:

```
The numbers 48375383 and 123456789 have the greatest common divisor 1
```

Quantum Period Finding as a QPE Routine

The core part of Shor's algorithm, is the period finding of the function $f(x) = a^x \bmod N$:

Find the minimal integer r such that: $a^r = 1 \bmod N$.

This can be computed via Quantum Phase Estimation (QPE) [\[4\]](#) (QPE was not discussed in the original formulation of Shor's algorithm but was later proposed by Kitaev [\[5\]](#)).

where $U|\psi_\theta\rangle = e^{2\pi i\theta}|\psi_\theta\rangle$ and $|\tilde{\theta}\rangle_m$ encodes an m -bit estimate of θ (in practice, the estimate $\tilde{\theta}$ is obtained by measurement, and the approximation holds with high probability up to an error of order $1/2^m$).

We can find the order r by applying QPE for the unitary

$$U_a|x\rangle = |ax \bmod N\rangle,$$

following the two facts (that are proven at the end of this notebook):

1. The unitary U_a has r eigenstates:

$$|\psi_s\rangle \equiv \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k / r} |a^k \bmod N\rangle \text{ with eigenvalues } \lambda_s = e^{2\pi i s / r}, \quad s = 0, 1, \dots, r - 1$$

2. We can easily prepare an equal superposition of $|\psi_s\rangle$, since

$$|1\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} |\psi_k\rangle.$$

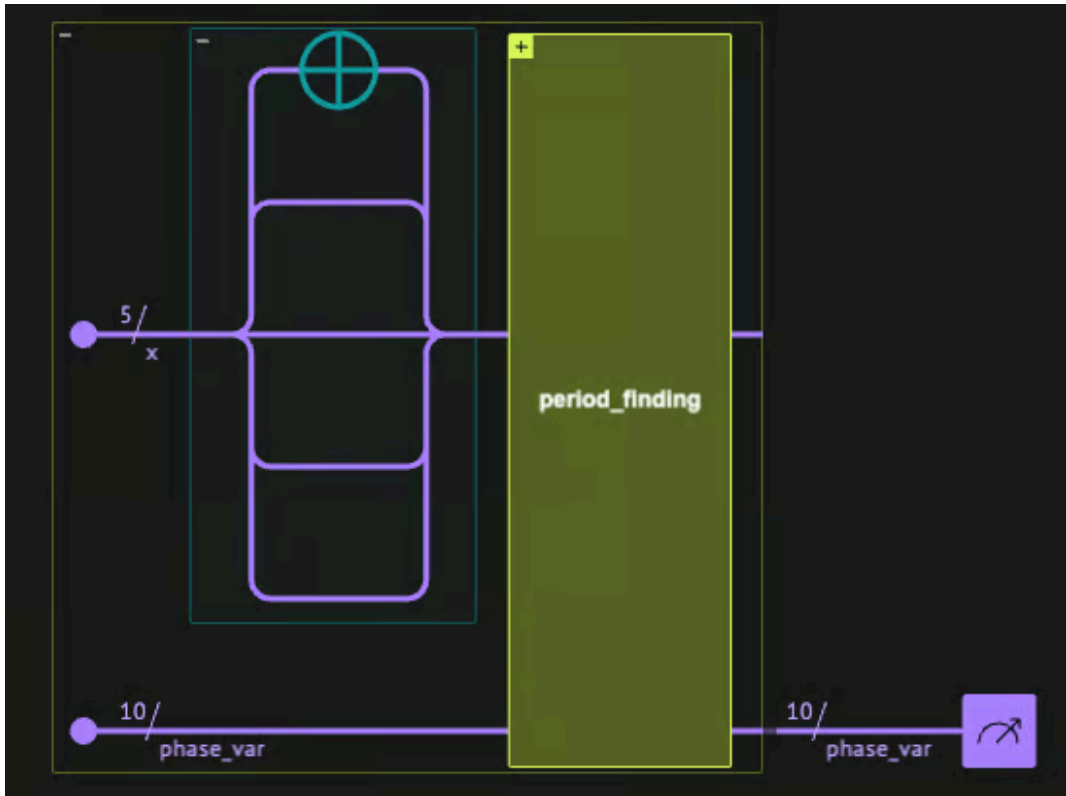
Therefore, by applying a QPE for U_a , on the state $|1\rangle$, we get an equal superposition for the estimation of the eigenphases $\{s/r\}_{s=0}^{r-1}$:

$$|1\rangle|0\rangle_m \xrightarrow{\text{QPE}(U_a)} \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |\psi_s\rangle |\tilde{s}/r\rangle_m.$$

How to extract the period r from the resulting state is shown in the following section. In this section we focus on the implementation of the quantum part. We work with the `qpe_flexible` function, that allows to pass a unitary with a specified powered operation. In particular, in our case we have

$$(U_a)^p |x\rangle = |a^p x \bmod N\rangle$$

```
def period_finding(n: CInt, a: CInt, x: QNum, phase_var: QNum):
    x ^= 1
    qpe_flexible(lambda p: modular_multiply_constant_inplace(n, a**p, x), phase_v
```



Postprocess with Continued Fractions Algorithm

The outcome distribution for the `phase_var` variable out of the QPE is in $[0, 1)$, and expected to be peaked around r values: s/r for $s = 0, 1, \dots, r - 1$. We can extract the value of r by using the continued fractions algorithm: any rational (irrational) number can be written as a finite (infinite) converging sequence of fractions:

$$x = a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \frac{1}{a_3 + \ddots}}}$$

There is an efficient classical algorithm for extracting a_i .

```
continued_fraction,
continued_fraction_convergents,
)

phase_value = Rational(17 / 1024)
list_of_continued_fraction = list(
    continued_fraction_convergents(continued_fraction(phase_value))
)
print(
    f"Continued fraction convergents for {phase_value}: {list_of_continued_fract
)
```

 Output:

```
Continued fraction convergents for 17/1024: [0, 1/60, 4/241, 17/1024]
```

Thus, we can extract r by taking a measurement of the `phase_var` variable, and calculate the series of rational numbers p_i/q_i that correspond to the continued fractions series. Then, we pick the fraction with the largest denominator q_i such that $q_i < N$, we should see, with high-probability, fractions that fit s/r for $s = 0, 1, \dots, r - 1$.

Let us define a function that applies those steps to a measurement result:

```
        continued_fraction_convergents(continued_fraction(Rational(phase_result))
    )
    # Eliminate fractions with denominator > n
    trimmed = [c for c in convs if c.as_numer_denom()[1] < n]
    # Store the last continued fraction_convergent
    lastf = trimmed[-1] if trimmed else None
    return lastf
```

Let us see an example, consider the following results out of the period finding algorithm, for $N = 18$:

```
phase_results = [0.1953125, 0.39160156, 0.0, 0.59667969, 0.80078125]
```

Our postprocess function gives

```
for phase_res in phase_results:
    print(f"{phase_res} --->

    {continued_fraction_post_process(phase_res, 18)}")
```

```
1/5
0.39160156 ---->

2/5
0.0 ---->

0
0.59667969 ---->

3/5
0.80078125 ---->

4/5
```

We can see that in this case one can conclude that $r = 5$ is the period we were trying to find.

Verifying the Order and Factoring

Once r is found, we first verify that it is an even number, this is because Shor's algorithm continues by writing

$$a^r = 1 \bmod N \implies (a^{r/2} - 1)(a^{r/2} + 1) = 0 \bmod N.$$

If r is even, this equation implies four scenarios:

1. $(a^{r/2} - 1) = 0 \bmod N$, impossible, since it contradicts the fact that r is the order.
2. $(a^{r/2} + 1) = 0 \bmod N$, in that case we must go back to step 1 and start with a new co-prime a .
3. $(a^{r/2} - 1)(a^{r/2} + 1) = N$, which gives a factoring for N .
4. $(a^{r/2} - 1)(a^{r/2} + 1) = kN$ with $k > 1$, which means that N divides the of the terms on the left. Thus, it has a non-trivial common divisor with one of them, namely, $\gcd(a^{r/2} +$

Below we define a function that verifies we are not in the second possibility, and returns the factors on N in case we are at the last two.

```
def get_factors(a, r, n):
    # r is odd
    if r % 2 == 1:
        print(f"The order r={r} is odd, return to the first step and find a co-prime")
        return None, None
    #  $a^{(r/2)} = -1 \pmod n$  is odd
    if a ^ (r // 2) + 1 % n == 0:
        print(
            f"It turns out that  $a^{(r/2)} + 1 \pmod N = 0$ , return to the first step and find a co-prime"
        )
        return None, None
    #  $(a^{(r/2)} + 1)(a^{(r/2)} - 1) = n$ 
    if (a ^ (r // 2) + 1) * (a ^ (r // 2) - 1) == n:
        return (a ^ (r // 2) + 1), (a ^ (r // 2) - 1)
    #  $(a^{(r/2)} + 1)(a^{(r/2)} - 1) = kn$ , with  $k > 1$ 
    gcd_ = np.gcd((a ^ (r // 2) + 1), n)
    if gcd_ > 1:
        return int(gcd_), int(n / gcd_)
    gcd_ = np.gcd((a ^ (r // 2) - 1), n)
    return int(gcd_), int(n / gcd_)
```

Example: Factoring 21

First, find a co-prime of N . We fix this value to $a = 11$ in order to get a full end-to-end result that runs the quantum part.

```
print(  
    f"The numbers {a_num} and {modulo_num} have the greatest common divisor {gcd_  
    )  
  
assert a_num == 11
```

ⓘ Output:

```
The numbers 11 and 21 have the greatest common divisor 1
```

Next, we run the quantum period finding algorithm to extract the period r .

Concerning the size of the quantum variables:

- For the phase variable that approximates the period r , we take twice as many qubits as in $|x\rangle$: this choice provides sufficient accuracy for extracting the period using the continued fraction algorithm (see the Technical Notes at the end of this notebook). In general, increasing the phase size yields deeper circuits with higher success probability of observing s/r , while decreasing it leads to shallower circuits with lower success probability.

```
@qtrunc
```

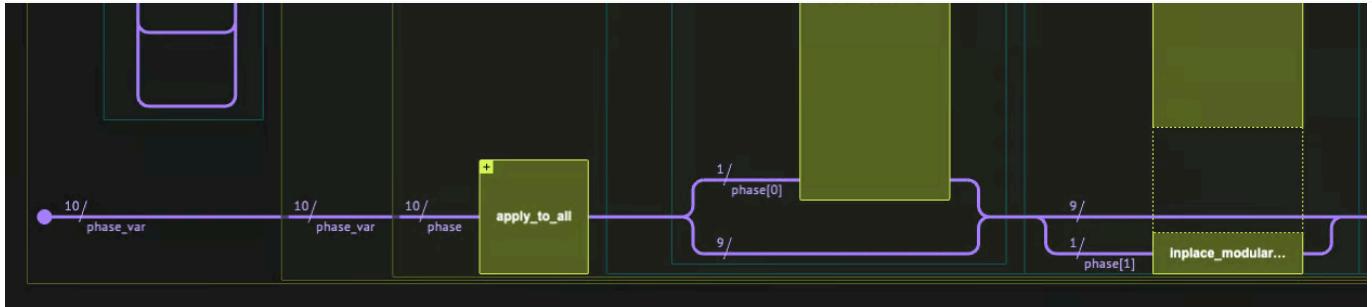
```
def main(phase_var: Output[QNum[phase_len, UNSIGNED, phase_len]]):  
    x = QNum()  
    allocate(x_len, x)  
    allocate(phase_var)  
    period_finding(modulo_num, a_num, x, phase_var)  
    drop(x)
```

```
qprog = synthesize(  
    main,  
    preferences=Preferences(qasm3=True, optimization_level=1),  
    constraints=Constraints(optimization_parameter="width"),  
)
```

```
show(qprog)
```

 Output:

Quantum program link: <https://platform.classiq.io/circuit/38AxlzzLp9PQW>



We execute and postprocess the results with the continued fraction approximation.

For the demonstration, let us postprocess all the results with more than 10 counts.

```
from IPython.display import display

result = execute(qprog).get_sample_result()
df = result.dataframe
display(df)
```

	phase_var	counts	count	probability	bitstring
0	0.000000	349	349	0.170410	0000000000
1	0.500000	347	347	0.169434	1000000000
2	0.833008	243	243	0.118652	1101010101
3	0.333008	238	238	0.116211	0101010101
4	0.166992	229	229	0.111816	0010101011
...	...	...	...	...	...
59	0.671875	1	1	0.000488	1010110000
60	0.673828	1	1	0.000488	1010110010
61	0.836914	1	1	0.000488	1101011001
62	0.843750	1	1	0.000488	1101100000
63	0.849609	1	1	0.000488	1101100110

64 rows × 5 columns

```
tracs.add(processed_trac)
print(f"For phase value {phase_res}: post processed fraction: {processed_frac")
```

Output:

```
For phase value 0.0: post processed fraction: 0
For phase value 0.5: post processed fraction: 1/2
For phase value 0.8330078125: post processed fraction: 5/6
For phase value 0.3330078125: post processed fraction: 1/3
For phase value 0.1669921875: post processed fraction: 1/6
For phase value 0.6669921875: post processed fraction: 2/3
For phase value 0.666015625: post processed fraction: 2/3
For phase value 0.833984375: post processed fraction: 5/6
For phase value 0.333984375: post processed fraction: 1/3
For phase value 0.166015625: post processed fraction: 1/6
For phase value 0.66796875: post processed fraction: 2/3
For phase value 0.6650390625: post processed fraction: 2/3
For phase value 0.83203125: post processed fraction: 5/6
For phase value 0.33203125: post processed fraction: 1/3
For phase value 0.16796875: post processed fraction: 1/6
```

We can see that the period is $r = 6$, as can be read from all the results that are post-processed to $1/6$ or $5/6$.

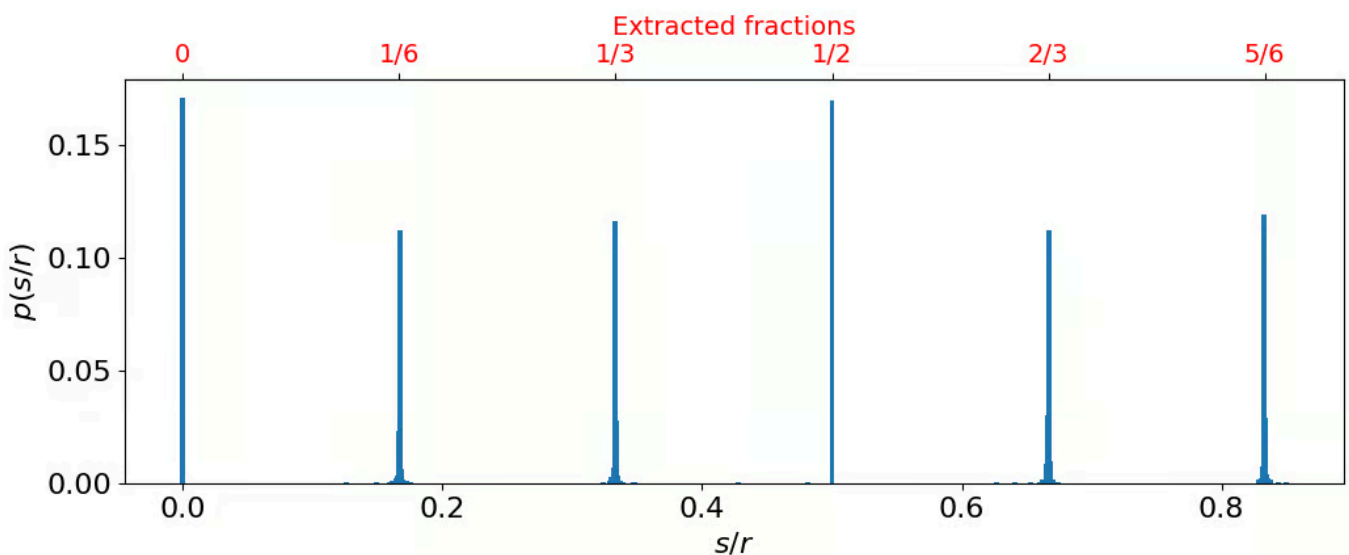
Let us plot the full distribution, together with the continued fraction approximation:

```
positions = [float(f) for f in fracs]
labels = ["0" if f == 0 else f"{f.numerator}/{f.denominator}" for f in fracs]
```

CLASSIQ

```
ax.bar(dtl["phase_var"], dtl["probability"], width=4 / 4**x_len)

ax.set_xlabel(r"$s/r$", fontsize=16)
ax.set_ylabel(r"$p(s/r)$", fontsize=16)
ax.tick_params(axis="both", labelsize=16)
# Extra top axis with the resulting continued fraction approximation
ax2 = ax.secondary_xaxis("top")
ax2.set_xticks(positions)
ax2.set_xticklabels(labels, color="red")
ax2.set_xlabel("Extracted fractions", fontsize=14, color="red")
ax2.tick_params(axis="both", labelsize=14);
```



We can clearly see that we have peaks at $\frac{0}{6}, \frac{1}{6}, \frac{2}{6}, \frac{3}{6}, \frac{4}{6}, \frac{5}{6}$, that is, the order is $r = 6$.

```
for p in positions:
    assert (p * 6).is_integer()
r = 6
```

All is left is to run the final step that determines the factor of N out of a and r :

```
factor1, factor2 = get_factors(a_num, r, modulo_num)
assert factor1 * factor2 == modulo_num
print(f"The number {modulo_num} is factored by {factor1} and {factor2}")
```

Technical Notes

The Eigenphases and Eigenstates of $f(x) = ax \bmod N$

Below we prove the claim that the modular multiplication by a , which is a co-prime of N , has the following eigenstates:

$$|\psi_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k / r} |a^k \bmod N\rangle, \quad s = 0, 1, \dots, r-1,$$

and the corresponding eigenvalues:

$$\lambda_s = e^{2\pi i s / r},$$

where r is the order of the function.

It is easy to verify that

$$a|\psi_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k / r} |a^{k+1} \bmod N\rangle = \frac{1}{\sqrt{r}} \sum_{k=1}^r e^{2\pi i s / r} e^{-2\pi i s k / r} |a^k \bmod N\rangle = e^{2\pi i s / r} \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k / r} |a^k \bmod N\rangle$$

where in the second equality we just changed the sum index $k \rightarrow k-1$, and the third one comes from the fact that the terms under the sum have a period of r , and the r -th term is equal to the 0-th one.

The Initial State $|1\rangle$ for the QPE

namely, the sum vanishes unless $k = 0$.

Calculating explicitly, we get

$$\frac{1}{\sqrt{r}} \sum_{k'=0}^{r-1} |\psi_{k'}\rangle = \frac{1}{r} \sum_{k'=0}^{r-1} \sum_{k=0}^{r-1} e^{-2\pi i k' k / r} |a^k \bmod N\rangle = \frac{1}{r} \sum_{k=0}^{r-1} \left(\sum_{k'=0}^{r-1} e^{-2\pi i k' k / r} \right) |a^k \bmod N\rangle = \frac{1}{r}$$

The Size of the Phase Variable

In Shor's algorithm, we choose the phase variable to be twice as large as the variable x on which we apply modular multiplication.

This choice follows from the properties of the QPE routine and the continued fractions algorithm.

Applying QPE with a phase variable of size m yields an m -bit approximation of the exact phase s/r :

$$\left| \frac{j}{2^m} - \frac{s}{r} \right| \leq \frac{1}{2^{m+1}}, \text{ for some } 0 \leq j \leq 2^{m-1}.$$

Now, according to the continued fractions algorithm, if we run the it for θ with $\left| \frac{s}{r} - \theta \right| \leq \frac{1}{2r^2}$, then both j and r can be recovered [3]. Thus, performing QPE with $m = 2n$ gives

$$\left| \frac{j}{2^m} - \frac{s}{r} \right| \leq \frac{1}{2^{2n+1}} \leq \frac{1}{2 \cdot N^2} \leq \frac{1}{2 \cdot r^2},$$

which satisfies the requirement of the continued fractions algorithm (the last two inequities comes from the fact the $r \leq N \leq 2^n$).

References

[1]: [Integer Factorization \(Wikipedia\)](#)

[2]: [P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring,"](#)

[Proceedings 35th Annual Symposium on Foundations of Computer Science \(FOCS\), IEEE, 1994.](#)

[5]: [Kitaev, A. Yu. "Quantum measurements and the Abelian Stabilizer Problem". arXiv:quant-ph/9511026 \(1995\).](#)

Was this page helpful?

 Yes

 No

[< Hidden-Shift Problem for Bent Functions](#)

[Hybrid Classical-Quantum Neural Network >](#)

Powered by [mintlify](#)